

Design — api-app foreground-service dataSync time-budget crash fix

- **Date:** 2026-06-14
- **Status:** DESIGN + PROTOTYPE — FOR OPERATOR REVIEW. **Do NOT ship without operator sign-off** (see §8).
- **Classification:** project-specific (Lava api-app Android FGS lifecycle).
- **Scope:** Defect 2 only from .lava-ci-evidence/crashlytics-resolved/2026-06-14-apiapp-0.2.6-10-field-events-investigation.md. Defect 1 (google-services.json mis-attribution) is OUT OF SCOPE here.
- **Branch:** prototype lives on the worktree branch; no version-file edits; no distribute.

1. Root cause (file:line, CONFIRMED from real Firebase data)

The api-app ships its embedded LAN API as a **long-lived foreground service** typed dataSync:

- api-app/src/main/AndroidManifest.xml:68 →
android:foregroundServiceType="dataSync"
- api-app/src/main/AndroidManifest.xml:20 → <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC"
>
- api-app/src/main/kotlin/lava/api/app/service/
ApiEngineService.kt:113 → startForeground(NOTIFICATION_ID, ...)
called UNCONDITIONALLY in onStartCommand, with **no try/catch**.

On Android 14+ (the field crashes are Android 16 / API 36 on a Galaxy S23 Ultra; the app's targetSdk = 35 per buildSrc/src/main/kotlin/lava/conventions/AndroidCommon.kt:15), dataSync (and mediaProcessing) are the **ONLY** foreground-service types subject to a **cumulative ~6-hour-per-24h runtime budget**. The api-app's embed is

structurally a persistent server — it is *meant* to run for many hours so peer devices on the LAN can reach it. It therefore exhausts the dataSync budget, and then:

1. The next `startForeground(dataSync)` (e.g. the user re-opens the app → `ApiControlViewModel.onStart()` → `serviceStarter.ensureRunning()` → `ContextCompat.startForegroundService` → `onStartCommand` → `startForeground`) throws **ForegroundServiceStartNotAllowedException: Time limit already exhausted for foreground service type dataSync** at `ApiEngineService.kt:113` → FATAL (Crashlytics issue `9ba8502ee0ba0d1fdd03987650b8acf8`, 3 events / 1 user).
2. The OS force-stops the over-running service → **RemoteServiceException\$ForegroundServiceDidNotStopInTimeException: A foreground service of type dataSync did not stop within its timeout** (Crashlytics issue `b9baeaede585fc3bc9b515c27cde532c`, 1 event / 1 user).

The service also never overrides `Service.onTimeout(int, int)` (added in API 35), so when the budget is hit there is no graceful-stop path — the OS internal-exception path is taken instead.

Why the current type is wrong, not just under-handled

dataSync is documented for "data transfer that the user is actively waiting on" with a bounded duration. A LAN API server that peers connect to at arbitrary times over an indefinite session is not a bounded sync — it is exactly the kind of long-lived-but-not-one-of-the-named-categories use case the platform provides specialUse for.

2. Authoritative Android facts (researched 2026-06-14; sources in §9)

Fact	Statement	Co
Time-budget types	ONLY dataSync and mediaProcessing are subject to the 6h/24h cumulative limit.	CO (d fg
onTimeout	At the 6h limit the system calls <code>Service.onTimeout(int, int)</code> (API 35+); the service has "a few seconds to call <code>Service.stopSelf()</code> ", else the system throws an internal exception.	CO

Fact	Statement	Co
specialUse time limit	NOT in the time-limited set → no cumulative cap.	CO lis ty sp
specialUse manifest	Requires <property android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE" android:value="..." /> inside <service>, plus FOREGROUND_SERVICE_SPECIAL_USE permission.	CO ty ex
specialUse Play review	"reviewed when you submit your app in the Google Play Console ... free-form ... provide enough information to let the reviewer see why you need" — free-form text justification.	CO ty
connectedDevice time limit	NOT time-limited.	CO
connectedDevice permission	FOREGROUND_SERVICE_CONNECTED_DEVICE + at least one of {CHANGE_NETWORK_STATE, CHANGE_WIFI_STATE, CHANGE_WIFI_MULTICAST_STATE , NFC, TRANSMIT_IR} OR a Bluetooth/UWB runtime permission. The api-app ALREADY declares CHANGE_WIFI_MULTICAST_STATE (manifest:18).	CO ty
connectedDevice Play review	Multiple third-party reports (e.g. Nordic DFU issue #424) that Play Console asks for a video justification for FOREGROUND_SERVICE_CONNECTED_DEVICE.	UI po de qu PE co PL flo co

3. Option analysis

Option	Removes 6h cap?	Requires	Play-policy n
A. Keep dataSync, wrap + reschedule (try/catch around startForeground, override onTimeout→stopSelf,	NO — cap stays; service still dies at 6h	small code change only	none

Option	Removes 6h cap?	Requires	Play-policy n
re-start when user foregrounds)			
B. specialUse	YES	manifest <property> + FOREGROUND_SERVICE_SPECIAL_USE perm + free-form Play justification	LOW (text justification; our use case "embedded local-network API server th peer devices the same LAN connect to" — a textbook specialUse) MEDIUM (UNCONFIRM video-justification requirement semantic fit i "interactions with externa devices" — a server others connect to is looser fit tha specialUse) HIGH — a lon running networking service WITHOUT an FGS type is exactly what Android 14+ restricts; the will throttle/
C. connectedDevice	YES	FOREGROUND_SERVICE_CONNECTED_DEVICE perm (the WiFi-multicast manifest perm we already have satisfies the secondary condition)	
D. Non-FGS persistent notification (drop FGS, run a plain bound/started service + ongoing notification)	N/A (no FGS = no cap)	large rework	

Option	Removes 6h cap?	Requires	Play-policy r
			it in the background
E. Restart-before-budget (proactively stop+restart the FGS just under 6h)	NO (still dataSync)	timer + lifecycle juggling	none

4. Recommendation

Adopt Option B (specialUse) as the foreground-service type, plus the defensive hardening from Option A layered underneath as belt-and-suspenders.

Rationale (safest path that keeps the API usable >6h without crashing and least Play-policy / breakage risk):

1. specialUse has **no cumulative time cap** → the embed can serve the LAN indefinitely, which is the product requirement.
2. specialUse's Play declaration is a **free-form text justification** (not a video), the lowest-friction review path, and our use case is a canonical "doesn't fit the other named types" server.
3. The defensive layer (try/catch around startForeground + an onTimeout override that stops gracefully) means that **even if** something unforeseen re-imposes a limit (OEM variance, a future policy change, a mis-declared variant), the app **degrades to a clean stop instead of a FATAL crash**. The crash site at line 113 is eliminated regardless of which type is in effect.
4. It does NOT touch the engine, the locks, the notification, mDNS, or the handoff provider — the change is confined to the FGS-type contract.

connectedDevice (Option C) is the fallback if Play review rejects the specialUse justification, but it carries the UNCONFIRMED video-justification risk and a looser semantic fit, so it is second choice.

5. Prototype (this branch)

Three files changed. **No version bump. No distribute.**

5.1 AndroidManifest.xml

- Replace `FOREGROUND_SERVICE_DATA_SYNC` permission with `FOREGROUND_SERVICE_SPECIAL_USE`.
- Change `android:foregroundServiceType="dataSync" → "specialUse"`.
- Add the mandatory `<property`
`android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE" ...>`
child describing the use case (the value is a string resource — the free-form Play-review justification string).

5.2 ApiEngineService.kt

- Wrap the `onStartCommand startForeground(...)` in a `try/catch` (`ForegroundServiceStartNotAllowedException`) so the launch can never FATAL at line 113; on catch, stop self gracefully (a `recordNonFatal` telemetry hook is marked TODO since the api-app does not yet wire an `AnalyticsTracker` here — §6.AC follow-up).
- Override `Service.onTimeout(startId, fgsType)` (guarded `Build.VERSION.SDK_INT >= 35`) to release locks + `stopForeground` + `stopSelf` within the few-second window, eliminating the `ForegroundServiceDidNotStopInTimeException` class.
- These are defensive and type-agnostic: they make the service crash-proof on the FGS-budget axis no matter which type ships.

5.3 strings.xml

- Add `api_fgs_special_use_justification` — the operator-editable Play-review justification copy referenced by the manifest `<property>`.

5.4 Prototype diff summary

AndroidManifest.xml

```
- <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC" />
+ <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" /
>
- android:foregroundServiceType="dataSync" />
+ android:foregroundServiceType="specialUse" >
```

```

+      <property
android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE"
+      android:value="@string/
api_fgs_special_use_justification" />
+ </service>

ApiEngineService.kt
+ import android.app.ForegroundServiceStartNotAllowedException
~ onStartCommand: startForeground(...) wrapped in try/catch →
graceful stop (no FATAL)
+ onTimeout(startId, fgsType): release locks + stopForeground +
stopSelf (SDK>=35)

strings.xml
+ api_fgs_special_use_justification

```

6. §6.AE Challenge plan (how to verify >6h survival)

The defect is a *time-budget* crash, so a literal 6h+ soak is the gold standard, but the budget can also be **simulated** so the gate is runnable inside the §6.AH container/VM emulator path.

1. **C-FGS-01 — type-is-not-dataSync structural test.** Parse the built/merged manifest; assert ApiEngineService's foregroundServiceType == "specialUse" AND the PROPERTY_SPECIAL_USE_FGS_SUBTYPE property is present AND FOREGROUND_SERVICE_DATA_SYNC is absent. Falsifiability: revert the manifest to dataSync → the test FAILS. (Runs on JVM; cheapest guard against silent type regression.)
2. **C-FGS-02 — onStartCommand never FATALs on budget-exhaustion.** Instrumented test on the emulator: make startForeground throw ForegroundServiceStartNotAllowedException (inject via a test seam / mock the OS boundary only) and assert the service stops gracefully rather than propagating the throw. Falsifiability: remove the try/catch → the test crashes the process / FAILS.
3. **C-FGS-03 — onTimeout graceful stop.** Instrumented test: invoke onTimeout(startId, type) and assert locks released + stopSelf reached within the window. Falsifiability: empty the onTimeout body → test FAILS (service still "running").
4. **C-FGS-04 — >6h survival (soak OR simulation).** Either (a) a real >6h device soak with the API reachable throughout (operator-run, captured logcat + periodic /health 200s), OR (b) a budget-simulation harness that drives the service past a shortened budget threshold on the emulator and asserts the API stays reachable. The simulation

is the §6.AE.7 honest-unblock path on darwin/arm64; the real soak is the operator gold-standard before ship.

All Challenge runs go through `scripts/run-api-app-challenge-matrix.sh` → Containers-submodule emulator per §6.AG / §6.AH (no host-direct, no live device). On this macOS host the run is honestly BLOCKED by §6.AH-debt until the container/VM emulator boots; that block is recorded, not bluffed past.

7. §6.O closure-log DRAFT

File (on ship): `.lava-ci-evidence/crashlytics-resolved/2026-06-14-apiapp-fgs-datasync-budget.md`

- **Crashlytics issues:** `9ba8502ee0ba0d1fdd03987650b8acf8` (FATAL `ForegroundServiceStartNotAllowedException`, `ApiEngineService.kt:113`) + `b9baeaede585fc3bc9b515c27cde532c` (`ForegroundServiceDidNotStopInTimeException`).
- **Stack-trace summary:** `dataSync` FGS exceeded the Android 14+ 6h/24h budget; next `startForeground(dataSync)` threw; over-running instance was OS-killed.
- **Root cause:** long-lived LAN API server typed `dataSync` (a time-budgeted type); see §1 above.
- **Fix commit SHA:** <filled on ship>.
- **Validation test:** C-FGS-01 (manifest type) + C-FGS-02 (no-FATAL) + C-FGS-03 (onTimeout) — paths <filled on ship>.
- **Challenge Test:** C-FGS-04 (>6h survival soak/simulation) — evidence path <filled on ship>.
- **Verification:** §6.AE matrix run on the Containers-submodule emulator; evidence <filled on ship>.
- **Note:** Defect 1 (google-services.json mis-attribution) tracked separately; after BOTH fixes ship, the api-app's crashes land on its OWN Firebase app (`d57b960e.../2932451e...`), not the client release app.

8. OPERATOR SIGN-OFF REQUIRED before ship

This branch is **design + prototype only**. Before this fix ships, the operator MUST:

1. **Choose the FGS type** — confirm `specialUse` (recommended) vs `connectedDevice` (fallback), accepting the Play-review path (free-form text for `specialUse`; possible video for `connectedDevice`).
2. **Confirm the Play Console `specialUse` justification copy** (the `api_fgs_special_use_justification` string value + the Console declaration) — this is operator-facing policy, not an agent decision.
3. **Author the C-FGS-01..04 tests** + run them through the Containers-submodule emulator gate (§6.AE / §6.AG / §6.AH) and capture real evidence — currently BLOCKED on this macOS host by §6.AH-debt; needs the Linux x86_64 KVM gate-host OR the container/VM TCG path.
4. **Bump `api-app` `versionCode`/`versionName`** per §6.Y (NOT done on this branch).
5. **Re-gate (§6.Z cold-start + C-FGS suite) + re-distribute the `api-app`** via the two-stage §6.AA flow — required for devices already running 0.2.6-10 to receive the fix.

No rebuild, no re-gate, no re-distribute happens without that sign-off. The prototype compiles (`:api-app:compileDebugKotlin` BUILD SUCCESSFUL) — that is necessary, NEVER sufficient (§6.Z): it has not been executed against a device.

9. Sources

- developer.android.com — Foreground service timeouts: <https://developer.android.com/develop/background-work/services/fgs/timeout>
- developer.android.com — Foreground service types: <https://developer.android.com/develop/background-work/services/fgs/service-types>
- developer.android.com — Android 15 behavior changes: <https://developer.android.com/about/versions/15/behavior-changes-15>
- developer.android.com — Changes to foreground service types for Android 15: <https://developer.android.com/about/versions/15/changes/foreground-service-types>

- Play Console Help — foreground service & full-screen intent requirements: <https://support.google.com/googleplay/android-developer/answer/13392821>
- Nordic DFU issue #424 (UNCONFIRMED connectedDevice video-justification report): <https://github.com/NordicSemiconductor/Android-DFU-Library/issues/424>
- Real Firebase data: `.lava-ci-evidence/crashlytics-resolved/2026-06-14-apiapp-0.2.6-10-field-events-investigation.md`